

Technische documentatie

ITProject Dreamteam 1 - LaboDashboard MVP

Flask - SQLite - Ansible - Docker - EVE-NG

Lina Ebrahim, Joost Stas, Bart De Pauw

Inhoudsopgave

1. Documentdoel en aansluiting op de opdracht	4
1.1 Kern van de MVP	4
1.2 MVP-afbakening, beperkingen en bewuste keuzes	5
2. Architectuur van de volledige opstelling	6
2.1 Architectuuroverzicht	6
2.2 Algemene werking van een configuratierun	6
2.3 Componenten en verantwoordelijkheden	7
2.4 Projectstructuur	7
3. Opstelling 1 – Basisopstelling	8
3.1 Netwerkschema	8
3.2 IP-adressering en poorten	8
3.3 Basisconfiguratie voor SSH	9
3.4 Routerconfiguratie via Ansible	9
3.5 Switchconfiguratie via Ansible	10
3.6 Servercontainers en tests via Ansible	10
3.7 Controlecommando's	11
4. Opstelling 2 - Podopstelling Brussel	12
4.1 Netwerkschema	12
4.2 IP-adressering	12
4.3 Basisconfiguratie voor SSH	13
4.4 Ansible-Playbook configuratie router-on-a-stick	14
4.5 Ansible-Playbook configuratie switches	14
4.6 Controlecommando's	15
5. Flask-applicatie	15
5.1 Gebruikte Python-packages	16
5.2 Structuur van de applicatie	16
5.3 Routes en webflow	17
5.4 Authenticatie en sessies	17
5.5 Dashboard, geschiedenis en backups	17
5.6 Aanpasbare variabelen en validatie	18
6. SQLite-database en deployment logs	19
6.1 Bestand database/schema.sql	19
6.2 Tabel users	19
6.3 Tabel network_setups	20
6.4 Tabel deployment_logs	20

6.5 Run reference en backups	21
6.6 Bestand modules/database_tools.py	21
7. Ansible algemeen	22
7.1 Brug tussen Flask en Ansible	22
7.2 Inventorybestanden	22
7.3 Ansible-Rollen	22
7.4 Variabelen in info.yml	23
7.5 Output, foutmeldingen en logging.....	23
7.6 Backups	23
8. Docker en servercontainers	24
8.1 Debian Docker-host	24
8.2 Dockerfiles	24
8.3 docker-compose.yml	24
8.4 Gebruikte poorten	25
8.5 Volumes	25
8.6 Netwerken	25
8.7 Buildproces	25
9. Installatiehandleiding Debian Docker-host.....	26
9.1 Debian installeren	26
9.2 Debian voorbereiden	27
9.3 Docker en Docker Compose installeren	27
9.4 Project ophalen	28
9.5 Docker-images bouwen en containers starten	28
9.6 SQLite initialiseren	29
9.7 Flask-applicatie openen	29
9.8 Demo-login en testflow	29
10. Bijlagen	30
10.1 Bijlage A - Projectverloop	30
10.2 Bijlage B – Scrummethode	31
10.3 Bijlage C - Logboeken en changelogs.....	32

1. Documentdoel en aansluiting op de opdracht

Het doel van dit project is een centrale beheeromgeving te bouwen voor docenten.

Een docent meldt zich aan in een Flask-webapplicatie, kiest een netwerkopstelling en start daarna een configuratie.

De Flask-applicatie start Ansible-playbooks die netwerktoestellen en serverdiensten configureren. De resultaten worden opgeslagen in SQLite en verschijnen opnieuw in het dashboard.

De volledige serverkant draait op een Debian Docker-host.

De Flask-applicatie, HTTP-server, HTTPS-server en FTP-server draaien als containers. De Docker-images worden zelf gebouwd en zijn gebaseerd op Alpine Linux.

De routers en switches worden geëmuleerd in EVE-NG en worden via SSH geconfigureerd.

1.1 Kern van de MVP

Onderdeel	Invulling in ons project
Webinterface	Flask-applicatie met loginpagina, dashboard, configuratieknop, output, geschiedenis en backupdownloads.
Database	SQLite-bestand data/itproject.db met users, network_setups en deployment_logs.
Automatisering	Ansible-playbooks voor routers, switches en servercontroles.
Serverdiensten	HTTP, HTTPS en FTP draaien als Docker-containers.
Netwerkkapparatuur	Cisco IOS-router(s) en switch(es) in EVE-NG, bereikbaar via management-IP en SSH.
Uitbreidingen	Twee opstellingen, logging per docent, configuratiegeschiedenis en backups per succesvolle run.

1.2 MVP-afbakening, beperkingen en bewuste keuzes

De opdracht laat toe om een MVP op te leveren. Daarom hebben we gekozen voor een duidelijke en reproduceerbare labo-opstelling die de kernwerking aantoont:

een docent kan aanmelden, een netwerkopstelling kiezen, een configuratie starten via Flask, Ansible voert de configuratie uit, SQLite bewaart de output en de resultaten zijn achteraf controleerbaar via logs en backups.

Niet elke mogelijke uitbreiding is uitgewerkt als productiefunctie. De onderstaande tabel toont welke keuzes bewust gemaakt zijn, waarom deze aanvaardbaar zijn voor de MVP en wat er nodig zou zijn voor een productieomgeving.

Keuze / beperking	Waarom dit aanvaardbaar is voor de MVP	Wat nodig is voor productie
Demo-credentials	Maakt de evaluatie eenvoudig en reproduceerbaar. De focus ligt op de technische werking van login, dashboard, Ansible en logging.	Unieke accounts per gebruiker, wachtwoordbeleid en eventueel MFA.
debug=True en vaste secret_key	Handig tijdens ontwikkeling en demo. Fouten zijn sneller zichtbaar en de setup blijft eenvoudig.	Debug uitschakelen en de secret key beheren via environment variables of secrets.
Docker socket mount in de Flask-container	Nodig om vanuit de Flask-container Docker-containers te kunnen starten en controleren.	Aparte worker, beperktere Docker API-rechten of een veiligere orkestratie-oplossing.
host_key_checking=False en oudere SSH-algoritmen	Nodig voor EVE-NG/Cisco IOS-labotoestellen die niet altijd moderne SSH-instellingen ondersteunen.	SSH-hardening, host key management en modernere netwerkimages.
Geen adminpagina om setups toe te voegen	Geen minimumvereiste. Setup1 en Setup2 zitten al in SQLite en in de projectbestanden, waardoor de demo reproduceerbaar blijft.	CRUD-interface met validatie, rechtenbeheer en controle op foutieve invoer.
Geen realtime Ansible-output	De output wordt wel volledig opgeslagen en na afloop getoond in het dashboard. Daarmee is de run controleerbaar.	Streaming via SSE, WebSocket of een job queue.
HTTPS niet op de Flask-applicatie zelf	HTTPS is wel aanwezig als aparte servercontainer. De Flask-applicatie draait in de MVP als interne labo-applicatie op poort 5000.	Reverse proxy met TLS-certificaat voor de Flask-applicatie zelf.
Geen aparte Ansible roles	De playbooks zijn bewust compact gehouden zodat de werking tijdens de evaluatie duidelijk uitlegbaar blijft.	Rollenstructuur met herbruikbare tasks, defaults, handlers en templates.
Setup2 configureert één pod	Eén pod toont het principe van de Brussel-podopstelling zonder onnodige complexiteit of poortduplicatie in de MVP.	Uitbreiden naar meerdere pods en volledige poortduplicatie volgens de fysieke labo-opstelling.

Belangrijk:

De punten in de kolom “Wat nodig is voor productie” betekenen niet dat de MVP fout is. Ze tonen welke extra maatregelen kunnen toegepast worden wanneer deze applicatie niet alleen in een demo-omgeving, maar in een echte school- of bedrijfsomgeving gebruikt zou worden.

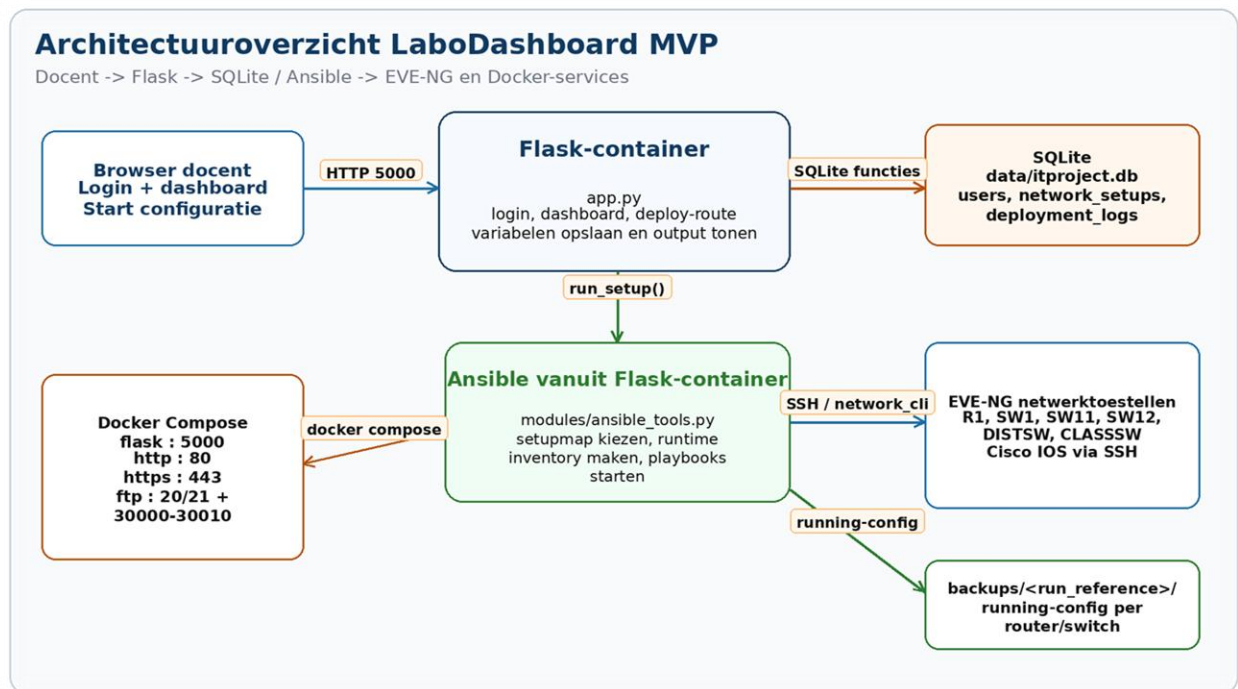
2. Architectuur van de volledige opstelling

De architectuur bestaat uit **vier lagen**:

- De browser van de docent,
- De Flask-container,
- De data laag met SQLite en backups,
- De uitvoeringslaag met Ansible, Docker Compose en tijdens onze demo: EVE-NG.

De docent werkt alleen met de webinterface. De configuratie zelf wordt door de backend automatisch uitgevoerd.

2.1 Architectuuroverzicht



Figuur 1 - Architectuur van de volledige LaboDashboard MVP.

2.2 Algemene werking van een configuratierun

1. De docent opent de Flask-applicatie op poort 5000 en meldt zich aan.
2. Het dashboard haalt de beschikbare netwerkopstellingen op uit SQLite.
3. De docent kiest een setup en klikt op Start configuratie.
4. Flask maakt een unieke run_reference aan en roept modules/ansible_tools.py aan.
5. De Ansible-helper kiest de juiste setupmap en maakt een runtime inventory in data/runtime_inventories/.
6. De playbooks worden uitgevoerd in de volgorde router.yml, switch.yml en indien aanwezig servers.yml.
7. stdout, stderr en de eigen samenvatting worden opgeslagen in deployment_logs.
8. Bij succesvolle netwerkconfiguratie worden running-configs opgeslagen in backups/<run_reference>/.
9. Het dashboard toont de status, technische output, geschiedenis en downloadbare backups.

2.3 Componenten en verantwoordelijkheden

Component	Rol in het project	Belangrijkste bestanden
Flask	Webapplicatie voor login, dashboard, configuratie starten, output tonen en backups downloaden.	app.py, templates/, static/
SQLite	Lokale opslag voor gebruikers, netwerkopstellingen en deployment logs.	database/schema.sql, data/itproject.db
Database-module	Alle databasebewerkingen apart van app.py houden.	modules/database_tools.py
Ansible-module	Setupmap zoeken, runtime inventory maken, playbooks starten en output samenvatten.	modules/ansible_tools.py
Ansible-playbooks	Router-, switch- en serverconfiguratie uitvoeren.	ansible/playbooks/setup1/, ansible/playbooks/setup2/
Docker Compose	Containers bouwen en starten.	docker-compose.yml, Dockerfile, servers/*/Dockerfile
EVE-NG	Routers en switches emuleren die via SSH worden geconfigureerd.	Externe labomgeving
Backups	Running-configs bewaren per configuratierun.	backups/<run_reference>/

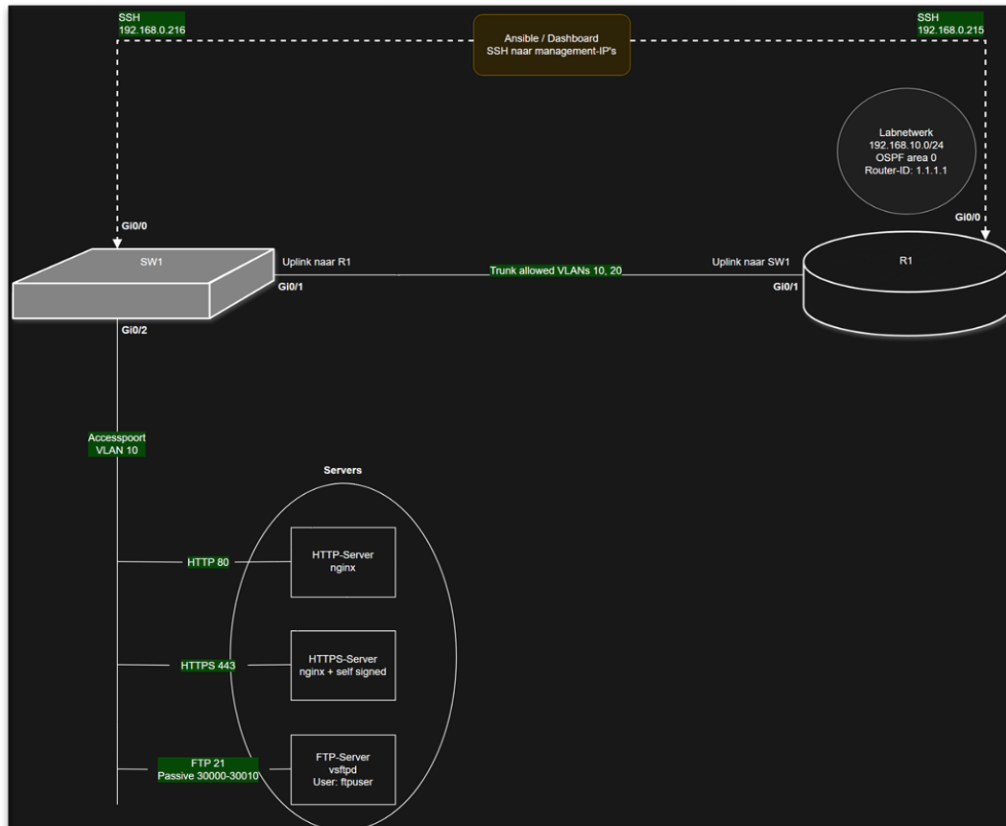
2.4 Projectstructuur

```
ITProject_Dreamteam1/
|-- app.py
|-- requirements.txt
|-- Dockerfile
|-- docker-compose.yml
|-- ansible.cfg
|-- ansible/
|   |-- playbooks/
|   |   |-- setup1/
|   |   |   |-- inventory.ini
|   |   |   |-- info.yml
|   |   |   |-- router.yml
|   |   |   |-- switch.yml
|   |   |   |-- servers.yml
|   |   |-- setup2/
|   |   |   |-- inventory.ini
|   |   |   |-- info.yml
|   |   |   |-- router.yml
|   |   |   |-- switch.yml
|-- database/
|   |-- schema.sql
|-- modules/
|   |-- database_tools.py
|   |-- ansible_tools.py
|-- servers/
|   |-- http/
|   |-- https/
|   |-- ftp/
|-- templates/
|-- static/
|-- data/
|-- backups/
|-- docs/
```

3. Opstelling 1 – Basisopstelling

Setup1 is de minimale bewijsopstelling. Deze setup toont dat het dashboard een router, een switch en drie serverdiensten kan aansturen. De router en switch draaien in de demo in EVE-NG. De HTTP-, HTTPS- en FTP-services draaien als Docker-containers op de Debian Docker-host.

3.1 Netwerkschema



Figuur 2 - Setup1 met Flask, servercontainers, R1 en SW1.

3.2 IP-adressering en poorten

Onderdeel	Waarde	Betekenis
R1 management	192.168.0.215	IP waarop Ansible R1 via SSH bereikt.
SW1 management	192.168.0.216	IP waarop Ansible SW1 via SSH bereikt.
R1 labinterface	GigabitEthernet0/1 - 192.168.10.1/24	Gateway/labinterface voor het labnetwerk.
VLAN 10	DOCENTEN	Access-VLAN op GigabitEthernet0/2.
VLAN 20	STUDENTEN	Extra VLAN dat via trunk GigabitEthernet0/1 wordt meegenomen.
HTTP	poort 80	nginx HTTP-testpagina.
HTTPS	poort 443	nginx met self-signed certificaat.
FTP	poort 20/21 + passive 30000-30010	vsftpd met ftpuser en test.txt.

3.3 Basisconfiguratie voor SSH

Ansible kan pas werken wanneer de toestellen bereikbaar zijn via SSH. Daarom krijgt elk toestel in EVE-NG eerst een kleine basisconfiguratie. Deze basisconfiguratie dient alleen om management, gebruiker, wachtwoord en SSH klaar te zetten. De echte laboconfiguratie gebeurt daarna via Ansible.

Basisconfiguratie R1	Basisconfiguratie SW1
<pre>hostname R1 no service config no ip domain lookup ip domain name itproject.local enable secret azerty username admin privilege 15 secret azerty interface GigabitEthernet0/0 description Management voor SSH en Ansible ip address 192.168.0.215 255.255.255.0 no shutdown ip ssh version 2 crypto key generate rsa modulus 1024 line vty 0 4 login local transport input ssh privilege level 15 exec-timeout 30 0 end write memory</pre>	<pre>hostname SW1 no service config no ip domain lookup ip domain name itproject.local enable secret azerty username admin privilege 15 secret azerty interface Vlan1 description Management voor SSH en Ansible no ip address dhcp no ip address ip address 192.168.0.216 255.255.255.0 no shutdown ip default-gateway 192.168.0.1 ip ssh version 2 crypto key generate rsa modulus 1024 line vty 0 4 login local transport input ssh privilege level 15 exec-timeout 30 0 end write memory</pre>

3.4 Routerconfiguratie via Ansible

Het playbook `ansible/playbooks/setup1/router.yml` configureert de router R1. De waarden komen uit `setup1/info.yml`. Daardoor staan IP-adressen, interface en OSPF-waarden niet hardcoded in de playbook zelf.

```
hostname R1
interface GigabitEthernet0/1
description Link naar LAN of switch
ip address 192.168.10.1 255.255.255.0
no shutdown
router ospf 1
router-id 1.1.1.1
network 192.168.10.0 0.0.0.255 area 0
write memory
```

Actie	Waarom
Hostname instellen	R1 is herkenbaar in de CLI en in de backup.
Labinterface configureren	De router krijgt een actief gateway-IP voor het labnetwerk.
OSPF basisconfiguratie	Toont dat routing via Ansible kan worden geplaatst.
write memory	De running-config wordt opgeslagen wanneer er wijzigingen zijn.
Backup maken	show running-config wordt bewaard onder backups/<run_reference>/.

3.5 Switchconfiguratie via Ansible

Het playbook `ansible/playbooks/setup1/switch.yml` configureert SW1. Het maakt VLANs aan, zet een accesspoort in VLAN 10 en configureert een trunkpoort die VLAN 10 en VLAN 20 mag transporteren. De poorten komen uit `setup1/info.yml`.

```
hostname SW1
vlan 10
  name DOCENTEN
vlan 20
  name STUDENTEN
interface GigabitEthernet0/2
  description Accesspoort VLAN 10
  switchport mode access
  switchport access vlan 10
  no shutdown
interface GigabitEthernet0/1
  description Trunk naar router of uplink
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan 10,20
  no shutdown
```

Actie	Waarom
Hostname instellen	SW1 is herkenbaar in de CLI en in de backup.
VLANs aanmaken	VLAN 10 DOCENTEN en VLAN 20 STUDENTEN worden aangemaakt zodat de switch verkeer logisch kan scheiden.
Accesspoort configureren	Gi0/2 wordt een accesspoort in VLAN 10. Een toestel op deze poort komt dus in het docentennetwerk terecht.
Trunkpoort configureren	Gi0/1 wordt een trunkpoort. Deze poort laat VLAN 10 en VLAN 20 door naar de router of uplink.
write memory	De running-config wordt opgeslagen wanneer er wijzigingen zijn.
Backup maken	show running-config wordt bewaard onder backups/<run_reference>/.

3.6 Servercontainers en tests via Ansible

Setup1 bevat ook `servers.yml`. Deze playbook start of controleert de Docker-containers voor HTTP, HTTPS en FTP. Hierdoor wordt aangetoond dat Ansible niet alleen netwerkapparatuur configureert, maar ook serverdiensten op de Docker-host kan controleren.

Service	Container	Poort	Controle in playbook
HTTP	itproject_http	80	GET naar http://http verwacht status 200.
HTTPS	itproject_https	443	GET naar https://https verwacht status 200; certificaatvalidatie staat uit door het self-signed certificaat.
FTP	itproject_ftp	20/21 + passive	Poort 21 bereikbaar; login met ftpuser/itproject; map ftp openen; test.txt vinden.

3.7 Controlemando's

Router:

```
show ip interface brief
show running-config interface GigabitEthernet0/1
show running-config | section router ospf
```

Switch:

```
show vlan brief
show running-config interface GigabitEthernet0/2
show running-config interface GigabitEthernet0/1
show interfaces trunk
```

Servers:

```
curl http://<DEBIAN-HOST-IP>
curl -k https://<DEBIAN-HOST-IP>
ftp <DEBIAN-HOST-IP>
```

4. Opstelling 2 - Podopstelling Brussel

Setup2 is de uitbreiding op het podverhaal uit de opdracht. In de volledige opdracht bestaat een rack uit meerdere pods. In onze MVP configureren we bewust 1 pod volledig. Zo tonen we het principe zonder de opstelling groter te maken dan nodig voor een verdedigbare demo.

Deze setup bevat R1, twee pod-switches, een distributieswitch en een classroomswitch. R1 werkt als router-on-a-stick. Tussen DISTSW en CLASSSW wordt EtherChannel gebruikt. Op CLASSSW worden voorbeeldpoorten voorzien voor docenten, studenten, servers en gasten.

4.1 Netwerkschema



Figuur 3 - Setup2 als MVP van een podopstelling met classroomzijde.

4.2 IP-adressering

Toestel / VLAN	IP-adres / waarde	Rol
R1	192.168.0.221	Router/gateway voor VLAN 10, 20, 30 en 40.
SW11	192.168.0.222	Eerste pod-switch.
SW12	192.168.0.223	Tweede pod-switch.
DISTSW	192.168.0.224	Distributieswitch aan podzijde.
CLASSSW	192.168.0.225	Classroomswitch aan studentenzijde.
VLAN 10	192.168.10.1/24 op R1-subinterface	VLAN 10 DOCENTEN
VLAN 20	192.168.20.1/24 op R1-subinterface	VLAN 20 STUDENTEN
VLAN 30	192.168.30.1/24 op R1-subinterface	VLAN 30 SERVERS
VLAN 40	192.168.40.1/24 op R1-subinterface	VLAN 40 GASTEN

4.3 Basisconfiguratie voor SSH

Ook bij setup2 moeten de toestellen eerst via SSH bereikbaar zijn. De startup-config in EVE-NG dient enkel voor management. De configuratie van VLANs, trunks, subinterfaces en EtherChannel gebeurt daarna door Ansible.

Basisconfiguratie R1	Basisconfiguratie Switches
<pre>hostname R1 no service config no ip domain lookup ip domain name itproject.local enable secret azerty username admin privilege 15 secret azerty default interface GigabitEthernet0/1 interface GigabitEthernet0/1 description Management voor SSH en Ansible ip address 192.168.0.221 255.255.255.0 no shutdown ip ssh version 2 crypto key generate rsa modulus 1024 line vty 0 4 login local transport input ssh privilege level 15 exec-timeout 30 0 end write memory</pre>	<pre>hostname <HOSTNAME> no service config no ip domain lookup ip domain name itproject.local enable secret azerty username admin privilege 15 secret azerty interface Vlan1 description Management voor SSH en Ansible no ip address dhcp no ip address ip address <MANAGEMENT-IP> 255.255.255.0 no shutdown ip default-gateway 192.168.0.1 ip ssh version 2 crypto key generate rsa modulus 1024 line vty 0 4 login local transport input ssh privilege level 15 exec-timeout 30 0 end write memory</pre>

Voor de switches gebruiken we dezelfde basisconfiguratie. Alleen hostname en management-IP verschillen per switch.

Switch	Hostname	Management-IP
SW11	SW11	192.168.0.222
SW12	SW12	192.168.0.223
DISTSW	DISTSW	192.168.0.224
CLASSSW	CLASSSW	192.168.0.225

4.4 Ansible-Playbook configuratie router-on-a-stick

De router gebruikt router-on-a-stick. De fysieke interface Gi0/0 krijgt geen IP-adres. Daarop worden subinterfaces gemaakt voor VLAN 10, VLAN 20, VLAN 30 en VLAN 40.

Ansible-configuratie R1	Wat de configuratie juist configureert
<pre>hostname R1 interface GigabitEthernet0/0 description Trunk naar SW11 voor VLAN 10, VLAN 20, VLAN 30 en VLAN 40 no ip address no shutdown interface GigabitEthernet0/0.10 encapsulation dot1Q 10 ip address 192.168.10.1 255.255.255.0 interface GigabitEthernet0/0.20 encapsulation dot1Q 20 ip address 192.168.20.1 255.255.255.0 interface GigabitEthernet0/0.30 encapsulation dot1Q 30 ip address 192.168.30.1 255.255.255.0 interface GigabitEthernet0/0.40 encapsulation dot1Q 40 ip address 192.168.40.1 255.255.255.0</pre>	- Hostname instellen; - Fysieke trunkinterface voorbereiden; - Subinterfaces maken voor VLAN 10, 20, 30 en 40; - Elke subinterface krijgt dot1Q encapsulation; - Elke VLAN krijgt een eigen gateway-IP; - Running-config ophalen en bewaren als backup.

4.5 Ansible-Playbook configuratie switches

De setup2-switchplaybook is dynamisch: dezelfde playbook draait op SW11, SW12, DISTSW en CLASSSW. Per inventory_hostname zoekt de playbook de juiste instellingen op in setup2/info.yml. Daardoor kan elke switch een andere rol krijgen zonder vier aparte playbooks te maken.

Switch	Configuratie via Ansible
Alle switches	VLAN 10 DOCENTEN, VLAN 20 STUDENTEN, VLAN 30 SERVERS en VLAN 40 GASTEN.
SW11	Gi0/1 trunk naar R1 en Gi0/2 trunk naar SW12.
SW12	Gi0/1 trunk naar SW11 en Gi0/2 trunk naar DISTSW.
DISTSW	Gi0/1 trunk naar SW12. Gi0/2 en Gi0/3 worden lid van EtherChannel Port-channel1.
CLASSSW	Gi0/1 en Gi0/2 worden lid van EtherChannel Port-channel1. Daarna worden Gi0/3, Gi0/4, Gi0/5 en Gi0/6 als accesspoorten gebruikt voor VLAN 10, VLAN 20, VLAN 30 en VLAN 40.

Voorbeeld uit de gegenereerde switchconfiguratie:

```
vlan 10
  name DOCENTEN
vlan 20
  name STUDENTEN
vlan 30
  name SERVERS
vlan 40
  name GASTEN

interface Port-channel1
  switchport trunk encapsulation dot1q
  switchport mode trunk
  switchport trunk allowed vlan 10,20,30,40
  no shutdown
```

4.6 Controlecommando's

```
Router:
show ip interface brief
show running-config interface GigabitEthernet0/0
show running-config interface GigabitEthernet0/0.10
show running-config interface GigabitEthernet0/0.20
show running-config interface GigabitEthernet0/0.30
show running-config interface GigabitEthernet0/0.40
```

```
Switches:
show vlan brief
show interfaces trunk
```

```
Alleen op DISTSW en CLASSSW:
show etherchannel summary
show running-config interface Port-channel1
```

5. Flask-applicatie

Voor dit project werd gebruikgemaakt van het Python-framework **Flask**.

Flask is een lichtgewicht webframework waarmee eenvoudig webapplicaties ontwikkeld kunnen worden. Het framework biedt ondersteuning voor routing, sessiebeheer, formulieren en het dynamisch genereren van webpagina's via templates.

Binnen dit project vormt Flask de centrale schakel tussen de gebruiker, de database en de Ansible-automatisering. Via een webinterface kunnen gebruikers zich aanmelden, netwerkopstellingen bekijken, configuraties starten en de resultaten opvolgen.

Flask verwerkt de gebruikersacties, haalt gegevens op uit de database en toont deze dynamisch in de HTML-pagina's.

Daarnaast wordt Flask gebruikt voor:

- het beheren van gebruikerssessies
- het opslaan van configuratiegeschiedenis
- het downloaden van backups
- het valideren van configuratievariabelen voordat deze worden toegepast op de netwerkopstellingen.

5.1 Gebruikte Python-packages

Voor de ontwikkeling van de Flask-applicatie werden verschillende Python-packages en voorgedefinieerde functies gebruikt om bepaalde functionaliteiten binnen de flask-applicatie te optimaliseren.

Package / module	Gebruik in het project
flask	Webframework voor login, dashboard, routes en templates.
werkzeug	Hashen en controleren van wachtwoorden.
ansible	Uitvoeren van playbooks vanuit de Flask-container.
paramiko==2.12.0	SSH-ondersteuning voor Ansible network_cli richting Cisco IOS in EVE-NG.
pyyaml	Lezen en schrijven van info.yml voor setupvariabelen.
datetime	Datum- en tijdstempels genereren voor logging
zoneinfo	Belgische tijdzone toepassen
pathlib	Werken met bestanden en mappen
sqlite3	Opslaan van gebruikers en configuratielogs

5.2 Structuur van de applicatie

De applicatie werd ontwikkeld volgens een modulaire structuur waarbij de verschillende onderdelen gescheiden werden gehouden.

Bestand / map	Functie
app.py	Start de Flask-app, initialiseert SQLite, beheert routes en sessies.
templates/login.html	Loginpagina voor docentaccounts.
templates/dashboard.html	Dashboard met setupkaarten, startknoppen, variabelen, output, geschiedenis en backups.
static/style.css	Eigen styling bovenop Bootstrap.
modules/database_tools.py	Databasefuncties voor users, setups, logs en backups.
modules/ansible_tools.py	Validatie van variabelen, runtime inventory en uitvoeren van playbooks.
ansible/playbooks/setup*/info.yml	Bron voor technische setupinformatie en aanpasbare waarden.

5.3 Routes en webflow

Route	Methode	Doel
/	GET/POST	Loginpagina tonen en ingevoerde credentials controleren.
/dashboard	GET	Dashboard tonen met beschikbare setups, laatste status en geschiedenis.
/deploy	POST	Gekozen setup starten via Ansible en deployment log opslaan.
/update-setup-variables	POST	Aangepaste waarden uit de frontend valideren en bewaren in info.yml.
/backup/<run_reference>/<filename>	GET	Backupbestand downloaden dat bij een specifieke run hoort.
/logout	GET	Sessie leegmaken en terugkeren naar loginpagina.

5.4 Authenticatie en sessies

Docenten loggen in met een gebruikersnaam en wachtwoord.

Het wachtwoord wordt niet in plaintext opgeslagen, maar als hash in de users-tabel. Tijdens het inloggen vergelijkt Flask het ingegeven wachtwoord met de opgeslagen hash.

Bij een correcte login worden de gebruikersnaam, user_id en rol in de sessie geplaatst.

Stap	Wat gebeurt er
Loginformulier	De docent vult username en password in.
Controle	verify_user() zoekt de gebruiker op in SQLite en controleert de wachtwoordhash.
Sessie	Bij succes bewaart Flask user_id, username en role in session.
Dashboardbeveiliging	Routes controleren of user_id aanwezig is in de sessie.
Logout	session.clear() verwijdert de actieve sessie.

5.5 Dashboard, geschiedenis en backups

Het dashboard is gebouwd om tijdens de demo snel te tonen wat de applicatie doet.

Het toont de naam van de aangemelde docent, de beschikbare netwerkopstellingen, de status van de laatste run, de technische output en een historiek van configuratieruns.

Succesvolle runs tonen downloadlinks naar de backupbestanden.

- Setupkaarten komen uit de network_setups-tabel.
- De technische uitleg per setup komt uit info.yml.
- De geschiedenis komt uit deployment_logs en kan gefilterd worden op setup, status en docent.
- De output wordt opgesplitst in een samenvatting en technische output, zodat het leesbaar blijft.
- Backupdownloads worden gekoppeld aan deployment_logs.run_reference.

5.6 Aanpasbare variabelen en validatie

De frontend kan waarden uit info.yml tonen en in beperkte, gecontroleerde vorm laten aanpassen. Voorbeelden zijn IP-adressen, VLAN-nummers en interfacenamen. Voor het wegschrijven voert modules/ansible_tools.py validatie uit, zodat foutieve waarden niet zomaar in de playbooks terechtkomen.

Type waarde	Validatie
IP-adres	Moet syntactisch geldig zijn volgens ipaddress.
VLAN	Moet een getal tussen 1 en 4094 zijn.
Interface	Mag niet leeg zijn en mag geen spaties bevatten.
OSPF / Port-channel	Moet een bruikbaar nummer zijn.

6. SQLite-database en deployment logs

SQLite is de lokale database van de applicatie.

Dit past bij de MVP omdat er geen aparte databaseserver nodig is.

De database wordt als bestand opgeslagen in de data-map en kan via een Docker-volume blijven bestaan tussen containerstarts.

6.1 Bestand database/schema.sql

database/schema.sql bevat de SQL-structuur van de database.

app.py maakt de tabellen dus niet hardcoded aan.

modules/database_tools.py leest dit schema bij het initialiseren van de database. Dat maakt de code beter leesbaar en makkelijker te onderhouden.

Tabel	Functie
users	Bewaren van de loginaccounts voor de applicatie.
network_setups	Bewaren van de beschikbare network-setups die de docent kan kiezen.
deployment_logs	Bewaren van de resultaten van elke configuratierun.

6.2 Tabel users

Veld	Betekenis
id	Uniek nummer van de gebruiker.
username	Gebruikersnaam waarmee wordt ingelogd.
password_hash	Gehashte versie van het wachtwoord. Het gewone wachtwoord staat niet in de database.
role	Rol van de gebruiker. In onze MVP is dit teacher.

De standaardgebruikers worden bij het initialiseren aangemaakt of bijgewerkt.

Voor de demo zijn docent en docent2 voorzien. Het demo-wachtwoord is docent123.

Omdat dit een MVP is, zijn deze accounts bedoeld voor de labo-evaluatie en niet voor productiegebruik.

Het wachtwoord van de gebruikers wordt gehasht, dat is veiliger omdat het echte wachtwoord niet leesbaar in de database staat.

6.3 Tabel network_setups

De tabel network_setups bevat de netwerkopstellingen die in het dashboard gekozen kunnen worden.

Veld	Betekenis
id	Uniek nummer van de setup.
name	Naam die in het dashboard wordt getoond.
description	Korte beschrijving van de opstelling.
playbook_data	Naam van de map onder ansible/playbooks/, bijvoorbeeld setup1 of setup2.

In onze MVP werken we met twee opstellingen:

Setup	Betekenis	playbook_data
Basisopstelling	1 router, 1 switch, HTTP, HTTPS en FTP.	setup1
Podopstelling Brussel	1 pod met router, 2 pod-switches, distributieswitch en classroomswitch.	setup2

6.4 Tabel deployment_logs

Elke keer dat een docent op Start configuratie klikt, wordt het resultaat van die run opgeslagen in deployment_logs. Daardoor kan achteraf worden aangetoond wie wat heeft gestart, wanneer dat gebeurde en wat de output was.

Veld	Betekenis
id	Uniek nummer van de logregel.
user_id	Welke gebruiker de configuratie gestart heeft.
setup_id	Welke netwerkopstelling gekozen is.
timestamp	Tijdstip waarop de configuratielog is aangemaakt.
status	success of failed.
output	Samenvatting en technische output van Ansible.
run_reference	Extra veld in ons project om backups aan de juiste log te koppelen.

6.5 Run reference en backups

Bij elke configuratierun maakt app.py een unieke run_reference aan. Die naam wordt opgeslagen in deployment_logs en wordt ook gebruikt als mapnaam onder backups/.

De database bewaart dus de verwijzing; de backupbestanden zelf blijven gewone tekstbestanden op de schijf.

```
Voorbeeld run_reference:  
run-20260604-193721-docent2-setup1  
  
Voorbeeld mapstructuur:  
backups/  
|-- run-20260604-193721-docent2-setup1/  
|   |-- r1-running-config.txt  
|   |-- sw1-running-config.txt  
|-- run-20260604-194123-docent2-setup2/  
|   |-- r1-running-config.txt  
|   |-- sw11-running-config.txt  
|   |-- sw12-running-config.txt  
|   |-- distsw-running-config.txt  
|   |-- classsw-running-config.txt
```

Zo kan de docent in de geschiedenis niet alleen de technische output bekijken, maar ook de backupbestanden downloaden die bij die specifieke run horen.

6.6 Bestand modules/database_tools.py

Het bestand modules/database_tools.py bevat de Python-functies die de applicatie gebruikt om met SQLite te werken.

Belangrijke taken van dit bestand zijn:

Functie	Doel
init_database()	Tabellen aanmaken op basis van schema.sql en standaarddata voorzien.
verify_user()	Login controleren met gebruikersnaam en wachtwoord.
get_network_setups()	Beschikbare netwerkopstellingen ophalen voor het dashboard.
get_setup_info()	info.yml van een setup lezen voor technische informatie en variabelen.
save_deployment_log()	Resultaat van een configuratierun opslaan in deployment_logs.
get_deployment_logs_for_user()	Geschiedenis tonen en filteren in het dashboard.
get_backup_files_for_run()	Backupbestanden zoeken die bij een run_reference horen.

Door deze databasecode in een aparte module te plaatsen, blijft app.py overzichtelijker.

app.py regelt vooral de webflow, ondertussen database_tools.py verantwoordelijk is voor alles wat met SQLite te maken heeft.

7. Ansible algemeen

De concrete router-, switch- en serverconfiguratie staat in de hoofdstukken van de opstellingen.

Dit hoofdstuk beschrijft de algemene Ansible-werking:

- Waar de inventories staan
- Hoe Flask de playbooks start
- Hoe variabelen gebruikt worden
- Hoe output en backups worden verwerkt.

7.1 Brug tussen Flask en Ansible

Flask voert niet rechtstreeks configuratiecommando's uit op de toestellen.

app.py roept run_setup() aan uit modules/ansible_tools.py.

Die helper bepaalt:

- De setupmap
- Maakt een runtime inventory
- Start daarna ansible-playbook als subprocess.

De technische stdout en stderr worden verzameld en teruggegeven aan Flask.

7.2 Inventorybestanden

Bestand	Gebruik
ansible/playbooks/setup1/inventory.ini	Bevat R1 en SW1 met hun management-IP en SSH-instellingen.
ansible/playbooks/setup2/inventory.ini	Bevat R1, SW11, SW12, DISTSW en CLASSSW in logische groepen.
data/runtime_inventories/<run_reference>-inventory.ini	Kopie per run. Hierdoor blijft duidelijk met welke inventory een run uitgevoerd werd.

7.3 Ansible-Rollen

Er zijn geen aparte Ansible roles gebruikt. Dat is een bewuste MVP-keuze.

De playbooks zijn compact gehouden, omdat de evaluatie vooral moet aantonen dat de Flask-applicatie Ansible correct kan starten en dat de configuratie uitlegbaar blijft.

De rolverdeling zit wel logisch in de bestanden:

- router.yml voor routers.
- switch.yml voor switches.
- servers.yml voor serverdiensten.

7.4 Variabelen in info.yml

Elke setup heeft een eigen info.yml.

Dat bestand bevat de technische waarden voor de playbooks en de informatie die het dashboard toont. Hierdoor staat de configuratie niet volledig hardcoded in de playbooks.

Voorbeeldgebruik in een playbook:

```
vars_files:  
  - info.yml
```

Daarna kan de playbook waarden gebruiken zoals:

```
{{ variables.router.hostname }}  
{{ variables.router.management_ip }}  
{{ variables.switches.trunk_allowed_vlans }}
```

7.5 Output, foutmeldingen en logging

Ansible genereert technische output. Onze helper zet daarboven een leesbare samenvatting. Die samenvatting en de volledige technische output worden samen opgeslagen in SQLite.

Op het dashboard wordt de output opgesplitst zodat de docent eerst snel de status ziet en daarna eventueel de technische details kan openen.

```
SAMENVATTING CONFIGURATIE  
[OK] router.yml is succesvol uitgevoerd.  
[OK] switch.yml is succesvol uitgevoerd.  
[OK] servers.yml is succesvol uitgevoerd.
```

```
TECHNISCHE OUTPUT
```

```
--- router.yml ---
```

```
ANSIBLE OUTPUT
```

```
...
```

7.6 Backups

Na een succesvolle netwerkconfiguratie voert Ansible show running-config uit op de router(s) en switch(es).

De output wordt opgeslagen in backups/<run_reference>/.

Daardoor kan elke succesvolle run achteraf gecontroleerd worden aan de hand van de effectieve configuratie die op het toestel stond.

8. Docker en servercontainers

Docker wordt gebruikt voor de centrale Flask-applicatie en voor de drie serverdiensten HTTP, HTTPS en FTP.

Alle images worden zelf gebouwd. De Flask-container is gebaseerd op python:3.12-alpine. De servercontainers gebruiken alpine:latest met nginx, openssl of vsftpd.

8.1 Debian Docker-host

De Docker-host is een Debian-machine. Op die host staan Docker Engine en Docker Compose.

De containers draaien dus niet rechtstreeks op de laptop van de gebruiker, maar op de Debian-host die als centrale server fungeert.

8.2 Dockerfiles

Dockerfile	Base image	Belangrijkste installatie	Doel
Dockerfile	python:3.12-alpine	openssh-client, sshpass, docker-cli, docker-cli-compose, Python packages	Flask-applicatie met Ansible en Docker CLI.
servers/http/Dockerfile	alpine:latest	nginx	HTTP-server op poort 80.
servers/https/Dockerfile	alpine:latest	nginx, openssl	HTTPS-server op poort 443 met self-signed certificaat.
servers/ftp/Dockerfile	alpine:latest	vsftpd	FTP-server met ftpuser en testbestand.

8.3 docker-compose.yml

docker-compose.yml beschrijft welke containers gebouwd en gestart worden.

De Flask-container krijgt volumes voor data en backups.

Daarnaast wordt de Docker socket gemount, zodat Ansible vanuit de Flask-container docker compose kan gebruiken om servercontainers te starten of te controleren.

```
Belangrijkste compose-services:
flask:
  build: .
  container_name: itproject_flask
  ports: 5000:5000
  volumes:
    - ./data:/app/data
    - ./backups:/app/backups
    - /var/run/docker.sock:/var/run/docker.sock

http: build: ./servers/http ports: 80:80
https: build: ./servers/https ports: 443:443
ftp: build: ./servers/ftp ports: 20:20, 21:21, 30000-30010
```

8.4 Gebruikte poorten

Poort	Container	Protocol / doel
5000	itproject_flask	Flask-webinterface.
80	itproject_http	HTTP via nginx.
443	itproject_https	HTTPS via nginx met self-signed certificaat.
21	itproject_ftp	FTP control channel.
20	itproject_ftp	FTP data channel.
30000 - 30010	itproject_ftp	Passive FTP-poorten.

8.5 Volumes

Volume / mount	Gebruik
./data:/app/data	Persistente SQLite-database en runtime inventories.
./backups:/app/backups	Persistente backups van running-configs.
/var/run/docker.sock:/var/run/docker.sock	Laat de Flask-container Docker op de host aansturen. Dit is handig voor de demo, maar niet geschikt als productiebeveiliging.

8.6 Netwerken

Docker Compose maakt standaard een projectnetwerk aan. Binnen dat netwerk kunnen de containers elkaar bereiken via servicenaam.

Daarom kan servers.yml bijvoorbeeld `http://http` en `https://https` gebruiken.

De router en switches zitten niet in dit Docker-netwerk; die worden via hun management-IP bereikt.

8.7 Buildproces

1. docker compose build leest docker-compose.yml en de Dockerfiles.
2. Docker bouwt de Flask-image en de drie serverimages vanaf Alpine-gebaseerde images.
3. docker compose up -d start de containers op de achtergrond.
4. docker compose ps controleert of flask, http, https en ftp de status Up hebben.

9. Installatiehandleiding Debian Docker-host

9.1 Debian installeren

Debian is het Linux-besturingssysteem waarop onze Docker-host draait. We gebruiken Debian omdat het stabiel, licht en geschikt is om serverdiensten zoals Docker op te draaien. Binnen dit project draait Debian dus niet de applicatie rechtstreeks, maar wel de Docker-containers van Flask, HTTP, HTTPS en FTP.

De installatie-ISO kan gedownload worden via de officiële Debian-website:

<https://www.debian.org/CD/netinst/>

Voor dit project gebruiken we:

debian-**<version>**-amd64-netinst.iso

Tijdens de installatie kiezen we Debian als serverinstallatie. We installeren dus geen grafische desktopomgeving. Bij de softwareselectie vinken we enkel deze onderdelen aan:

- SSH server
- standard system utilities

Installatie Op Een Fysiek Toestel

Bij een fysiek toestel wordt Debian via een USB-stick geïnstalleerd:

- De Debian ISO wordt op een USB-stick gezet, bijvoorbeeld met Rufus.
- Daarna wordt het toestel vanaf deze USB-stick opgestart en wordt Debian op de interne schijf geïnstalleerd.

Na de installatie moet het toestel bereikbaar zijn via het netwerk. Daarom is een vast IP-adres of DHCP-reservatie handig. Via dit IP-adres kunnen we later met SSH verbinden en Docker beheren.

Installatie In Een Virtuele Machine	Onderdeel	Minimum specs	Aanbevolen specs
Voor testen of ontwikkeling kan Debian ook in een VM draaien. Dit kan bijvoorbeeld met VirtualBox, VMware of Proxmox.	CPU	2 cores	4 cores
	RAM	4 GB	6 GB
	Schijf	25 GB	40 GB
	Netwerk	Bridged adapter	
	Desktopomgeving	Niet installeren.	

De bridged adapter geeft de VM een eigen IP-adres op hetzelfde netwerk als de laptop. Daardoor kunnen we de Debian-host via SSH bereiken en de Flask-app testen alsof het een apart toestel is.

Wanneer de VM via NAT draait, moeten er poortdoorverwijzingen ingesteld worden. bijvoorbeeld zo:

- SSH: hostpoort 11022 naar gastpoort 22
- FLASK: hostpoort 5000 naar gastpoort 5000

Alle stappen in dit hoofdstuk vanaf hier worden uitgevoerd op de Debian Docker-host. Dit is de machine waarop Docker geïnstalleerd wordt en waarop de containers van het project draaien.

9.2 Debian voorbereiden

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git curl ca-certificates gnupg
```

Commando / pakket	Doel
sudo apt update	Meest recente pakketlijsten ophalen.
sudo apt upgrade -y	Beschikbare updates installeren.
git	Project van GitHub ophalen.
curl	Bestanden via terminal downloaden.
ca-certificates en gnupg	Officiële Docker-repository veilig toevoegen.

9.3 Docker en Docker Compose installeren

1. De map aangemaakt waarin de Docker-sleutel wordt bewaard:

```
sudo install -m 0755 -d /etc/apt/keyrings
```

2. De officiële Docker GPG-sleutel wordt opgehaald en opgeslagen.

```
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

3. De juiste leesrechten worden toegekend aan de sleutel.

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

4. Daarna voegen we de officiële Docker-repository toe aan Debian.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

5. Na het toevoegen van de repository moeten we de pakketlijst opnieuw updaten.

```
sudo apt update
```

6. Docker, Docker Compose en de nodige Docker-onderdelen installeren en activeren

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable docker
sudo systemctl start docker
```

7. De installatie controleren:

```
docker --version
docker compose version
```

Wanneer beide commando's een versienummer tonen, is Docker correct geïnstalleerd.

9.4 Project ophalen

Het project wordt opgehaald van GitHub met git clone.

```
git clone https://github.com/B-a-r-t-dp/ITProject_Dreamteam1.git
cd ITProject_Dreamteam1
```

9.5 Docker-images builden en containers starten

Commando / pakket	Doel
<i>sudo docker compose build --no-cache flask</i>	Dit commando leest docker-compose.yml en de bijhorende Dockerfiles. Op basis daarvan wordt de image aangemaakt voor de Flask-applicatie. De Container wordt hiermee nog niet gestart.
<i>sudo docker compose up -d flask</i>	De optie -d zorgt ervoor dat de containers op de achtergrond blijven draaien.
<i>sudo docker compose ps</i>	De containers voor Flask, HTTP, HTTPS en FTP moeten de status Up hebben. De Flask-applicatie is daarna bereikbaar op poort 5000 van de Debian Docker-host
<i>sudo docker compose down --remove-orphans</i>	Dit commando stopt en verwijdert de containers en netwerken van de Docker Compose-opstelling. Met --remove-orphans worden ook oude containers verwijderd die niet meer in docker-compose.yml staan.

9.6 SQLite initialiseren

De applicatie gebruikt SQLite om gebruikers, netwerkopstellingen en deployment logs op te slaan. Indien nodig kan de database manueel geïnitieerd worden met:

```
docker compose exec flask python -c "from modules.database_tools import init_database; init_database()"
```

Dit commando voert de functie `init_database()` uit binnen de Flask-container. Deze functie maakt de nodige tabellen aan en vult de basisgegevens aan.

9.7 Flask-applicatie openen

Wanneer de containers actief zijn, kan het dashboard geopend worden via een browser.

```
http://<IP-VAN-DEBIAN-HOST>:5000
```

Voorbeeld:

<http://192.168.0.200:5000>

Als de installatie correct verlopen is, verschijnt de loginpagina van het LaboDashboard.

Van daaruit kan de gebruiker zich aanmelden, een netwerkopstelling kiezen en de configuratie starten.

9.8 Demo-login en testflow

Gegeven	Waarde
Gebruiker 1	docent
Gebruiker 2	docent2
Demo-wachtwoord	docent123

1. Log in met een docentaccount.
2. Controleer dat de twee netwerkopstellingen zichtbaar zijn.
3. Start eerst Setup1 en controleer output, HTTP, HTTPS, FTP en backups.
4. Start daarna Setup2 en controleer router-on-a-stick, VLANs, trunks en EtherChannel.
5. Open de geschiedenis en toon dat de runs, status, output en backupbestanden bewaard zijn.

10. Bijlagen

10.1 Bijlage A - Projectverloop

Voor de realisatie van het project hebben we ons gebaseerd op de basisprincipes van Scrum. Daarnaast hebben we binnen het team duidelijke afspraken gemaakt over hoofddomeinen, bestandsstructuur en de manier waarop de verschillende onderdelen met elkaar moesten samenwerken.

We hebben bewust per persoon een hoofddomein gekozen.

Persoon	Hoofddomein	Belangrijkste bestanden	Verantwoordelijkheid
Lina	Flask / frontend	app.py, templates/login.html, templates/dashboard.html, static/	Login, logout, dashboardweergave, startknop, status/output, geschiedenis, backups in de interface en visuele documentatie.
Joost	SQLite / backend	database/schema.sql, modules/database_tools.py, data/	Database-initialisatie, gebruikerscontrole, network_setups, deployment_logs, timestamps, filters en validatie van setupwaarden.
Bart	Ansible / Docker / netwerk	modules/ansible_tools.py, ansible/playbooks/, Dockerfile, docker-compose.yml, servers/	Playbooks, inventories, Dockerfiles, servercontainers, EVE-NG-configuratie, router/switchconfiguratie en backupflow.

Deze manier van werken zorgde ervoor dat iedereen grotendeels zelfstandig kon verder werken aan zijn of haar sprinttaken zonder afhankelijk te zijn van andere personen. Dat betekent niet dat iedereen enkel zijn eigen hoofddomein moest kennen. De essentie was wel dat ieder van ons de volledige flow en de belangrijkste code in het project begrijpt.

Omdat de onderdelen sterk met elkaar verbonden zijn, hebben we op voorhand telkens duidelijke koppelafspraken gemaakt. Door die afspraken konden we apart werken aan Flask, SQLite, Ansible en Docker, maar bleef het project toch één samenhangend geheel over alle sprints heen.

Koppelpunt	Afspraak	Waarom dit belangrijk is
SQLite-tabellen	users, network_setups en deployment_logs blijven vaste tabelnamen.	Flask en database_tools.py gebruiken dezelfde structuur als schema.sql.
Ansible-resultaat	run_setup() geeft altijd een dictionary terug met status en output.	Flask kan elk resultaat op dezelfde manier tonen en opslaan in SQLite.
Statuswaarden	Alleen success of failed, in kleine letters.	Dashboardfilters en deployment_logs blijven voorspelbaar.
Setupdata	SQLite bepaalt welke setup bestaat; info.yml beschrijft technische waarden voor die setup.	Frontend en playbooks gebruiken dezelfde bron voor setupinformatie.
Backups	Elke run krijgt een run_reference en backups worden per runmap bewaard.	Geschiedenis, output en running-configs blijven achteraf koppelbaar.

10.2 Bijlage B – Scrummethode

Voor het project hebben we gewerkt met de basisprincipes van Scrum. We gebruikten een product backlog om alle taken te verzamelen en verdeelden die taken per sprint. Per sprint bepaalden we een duidelijk doel, kozen we taken uit de backlog en hielden we korte stand-ups om de voortgang op te volgen.

Tijdens de stand-ups bespraken we telkens wat al gedaan was, wat de volgende stap was en waar iemand eventueel vastzat. Op het einde van elke sprint bekeken we welke taken effectief klaar waren en welke taken eventueel moesten doorschuiven. Daarnaast maakten we retrospectives om kort te noteren wat goed ging, wat moeilijk liep en wat we meenamen naar de volgende sprint.

Sprint	Doel	Belangrijkste resultaat
Sprint 1	Basisstructuur en taakverdeling klaarzetten.	Projectstructuur, login/dashboardbasis, SQLite-schema, Dockerbestanden en eerste Ansible-playbooks stonden klaar.
Sprint 2	De basisopstelling technisch werkend maken.	Flask kon Ansible starten, output werd opgeslagen in SQLite, HTTP/HTTPS/FTP werden getest en router/switch-backups werden voorzien.
Sprint 3	MVP bruikbaar afwerken en tweede opstelling toevoegen.	Setup2 werd toegevoegd, variabelen werden aanpasbaar, geschiedenis en backups werden zichtbaar en de volledige flow werd verder getest.
Sprint 4	Oplevering, documentatie en demo voorbereiden.	Technische documentatie, code-uitleg, demo-flow, Scrum-documenten, changelogs en AI-logboeken werden afgewerkt.

Onze Scrumdocumenten bestaan uit:

Bronbestand	Inhoud	Gebruik in deze bijlage
docs/scrum/product-backlog.md	Product backlog met must-have, could-have en bewust niet-opgenomen onderdelen.	Gebruikt voor de backlogstatus en MVP-afbakening.
docs/scrum/sprint-1.md	Sprintplanning, backlogtaken, dailies en werkende sprintversie voor de basis.	Gebruikt voor Sprint 1: projectstructuur, login, SQLite, Ansible- en Dockerbasis.
docs/scrum/sprint-2.md	Sprintplanning voor integratie en uitbreiding van setup1.	Gebruikt voor Sprint 2: deployknop, serverchecks, EVE-NG-tests, output en backups.
docs/scrum/sprint-3.md	Sprintplanning voor setup2, geschiedenis, variabelen en backups.	Gebruikt voor Sprint 3: tweede opstelling, multi-setupflow en dashboardgeschiedenis.
docs/scrum/sprint-4.md	Sprintplanning voor oplevering, code learning, documentatie en demo.	Gebruikt voor de eindfase en de verdeling van documentatieonderdelen.
docs/scrum/retrospective-sprint-1.md	Retrospective met wat goed ging, moeilijk liep en actiepunten voor Sprint 2.	Gebruikt voor de leerpunten rond helperbestanden, EVE-NG en SSH.
docs/scrum/retrospective-sprint-2.md	Retrospective na integratie van de basisflow.	Gebruikt voor de leerpunten rond FTP, outputleesbaarheid en timestamps.
docs/scrum/retrospective-sprint-3.md	Retrospective na setup2 en de dashboarduitbreidingen.	Gebruikt voor de leerpunten rond demo, documentatie en code learning.
docs/opstartdocument.md	Opstartafspraken, MVP-scope en bestandseigenaars.	Gebruikt voor de teamverdeling en samenwerking.
docs/mvp-afbakening.md	Formele afbakening van wat wel, beperkt en niet in de MVP zit.	Gebruikt voor de verantwoording van bewuste keuzes.
docs/koppelafspraken.md	Technische afspraken tussen Flask, SQLite, Ansible en Docker.	Gebruikt om aan te tonen dat de onderdelen op vaste interfaces samenwerken.

10.3 Bijlage C - Logboeken en changelogs

Naast de Scrumdocumenten hield elk teamlid ook een persoonlijk AI-logboek en changelog bij.

Het AI-logboek werd gebruikt om kort te noteren wanneer AI als hulpmiddel werd gebruikt. Dit kon bijvoorbeeld gaan over debugging, documentatie, code-uitleg, foutanalyse of het herwerken van tekst in mensentaal.

De changelog werd gebruikt om per persoon bij te houden welke wijzigingen of beslissingen tijdens het project gemaakt werden. Daardoor blijft achteraf duidelijk wie aan welke onderdelen heeft gewerkt en waarom bepaalde aanpassingen zijn gebeurd.

Persoon	AI-logboek	Changelog	Hoofdfocus
Lina	docs/personen/lina/ai-logboek.md	docs/personen/lina/changelog.md	Flask/frontend, dashboard, outputweergave, geschiedenis, backups en visuele documentatie.
Joost	docs/personen/joost/ai-logboek.md	docs/personen/joost/changelog.md	SQLite/backend, databasefuncties, gebruikerscontrole, deployment logs, validatie en setupwaarden.
Bart	docs/personen/bart/ai-logboek.md	docs/personen/bart/changelog.md	Ansible, Docker, EVE-NG, setup1, setup2, servercontainers, backups en technische documentatie.

Deze logboeken zijn geen vervanging voor Git of de code zelf. Ze dienen vooral als extra verantwoording bij het projectverloop. Ze tonen hoe we problemen hebben onderzocht, welke keuzes we gemaakt hebben en hoe AI gebruikt werd als ondersteuning tijdens het project.

De effectieve codewijzigingen blijven zichtbaar via Git commits en de projectbestanden zelf.

De logboeken en changelogs geven daarbij extra context in mensentaal.